

ITS 365 – Project Phase 2 – CIFAR-10

By Marcus Nunez and Hakeem Atanda

I. Project Goals & Differences to Phase 1

The goal Phase 2 was to improve upon the results of Phase 1's analysis of the CIFAR-10 image dataset by applying and tweaking aspects of transfer learning. The previous custom CNN model trained only on the provided dataset was replaced by a pre-trained ResNet18, chosen due to instructor suggestion and reputation as a well-supported transfer learning model.

II. Data Processing and Training

The updated model utilizes the same dataset import process as in Phase 1, extracting the source file with 'zipfile', instead being resized exclusively to 224x224 to match the required input of ResNet18. This is the same reasoning for the replacement of manual to ImageNet normalization and the classification formula to reduce ImageNet to CIFAR-10's 10 classes.

```
dataset_dir = os.path.join(extracted_dir, 'CIFAR-10-images-master')
test_dir = os.path.join(dataset_dir, 'test')
train_dir = os.path.join(dataset_dir, 'train')

full_dataset_train = torchvision.datasets.ImageFolder(train_dir, transform=train_transform)
full_dataset_eval = torchvision.datasets.ImageFolder(train_dir, transform=eval_transform)
dataset_test = torchvision.datasets.ImageFolder(test_dir, transform=eval_transform)
```

The freezing of pre-trained layers was done due to now only needing to train on CIFAR-10 contents specifically, greatly improving speeds. This gained speed was unfortunately lost following the addition of another separated pipeline, giving training and evaluation exclusive transforms for improved generalization. The final model selection was also tweaked to include a comparison of validation loss to base further training off of. Together, these modifications (and mostly thanks to work handled by the pre-trained model) allowed for a massive reduction of 50 to 10 epochs with much more favorable results.

```
# transform

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(224, padding=16),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.1, contrast=0.1, saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])

#seperate evaluation transform excluding augmentation
eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

```

# init model
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
model.fc = nn.Linear(model.fc.in_features, 10)

criterion = nn.CrossEntropyLoss() # set CEL

# freeze to only train classifier
for param in model.parameters():
    param.requires_grad = False

for param in model.layer4.parameters():
    param.requires_grad = True

for param in model.fc.parameters():
    param.requires_grad = True

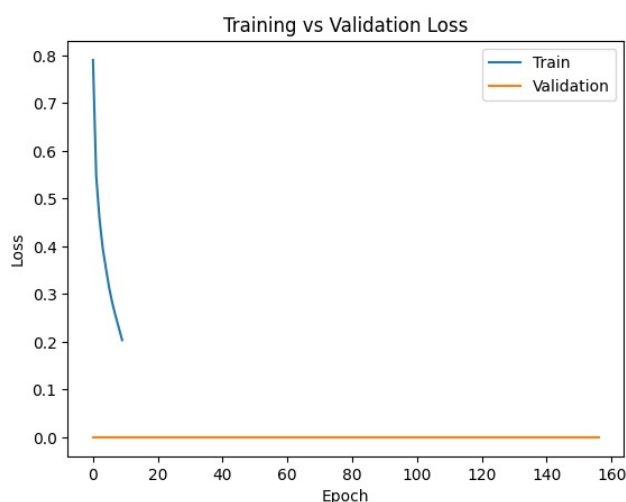
optimizer = optim.Adam([
    {"params": model.layer4.parameters(), "lr": 1e-4},
    {"params": model.fc.parameters(), "lr": 1e-3}
])

# ensure model uses gpu
model = model.to(device)

```

III. Results

The transfer learning model shows substantial improvement over its derivative, though boasting around the same speed. Phase 1's model previously achieved an F1 score and accuracy of ~72, whereas the updated model climbed to an F1 score of ~84 and an accuracy of ~84% (~87 was observed, but evidence could not be reproduced timely). As for the classification report, previous confusion in classes such as between cats and dogs still persists, with the new contender of deer and horses applying to the improved model.

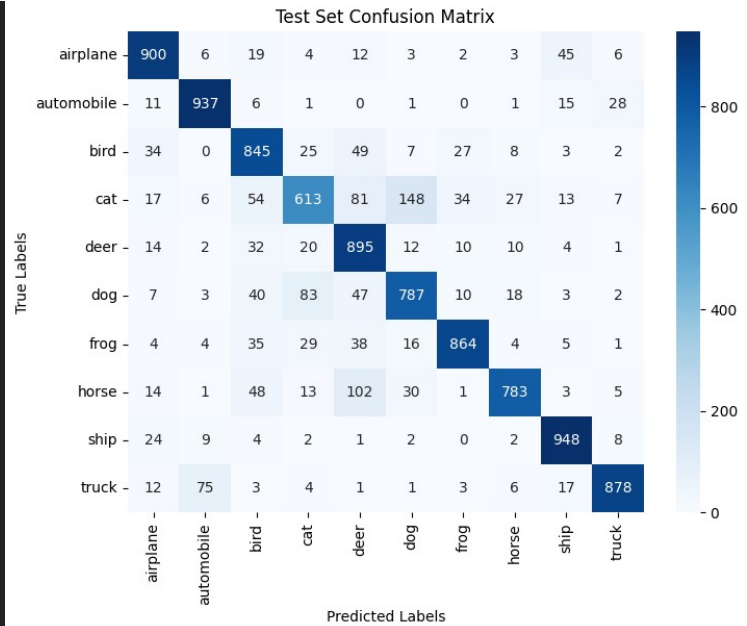


Validation Set Evaluation:					
	precision	recall	f1-score	support	
0	0.87	0.92	0.89	1000	
1	0.91	0.96	0.93	1000	
2	0.77	0.84	0.80	1000	
3	0.80	0.66	0.72	1000	
4	0.75	0.91	0.82	1000	
5	0.79	0.79	0.79	1000	
6	0.92	0.85	0.88	1000	
7	0.90	0.78	0.84	1000	
8	0.91	0.94	0.92	1000	
9	0.95	0.88	0.91	1000	
accuracy			0.85	10000	
macro avg			0.86	0.85	10000
weighted avg			0.86	0.85	10000
Validation Confusion Matrix:					
[[917 5 18 4 7 0 1 3 36 9]					
[4 957 5 3 1 1 0 1 10 18]					
[37 2 841 24 49 15 18 9 5 0]					
[15 6 52 663 59 128 28 34 10 5]					
[8 1 32 10 905 15 9 12 5 3]					
[10 2 47 73 37 794 10 19 4 4]					
[7 5 51 33 34 20 846 2 2 0]					
[16 0 35 15 110 28 2 782 7 5]					
[23 11 7 1 3 1 2 1 944 7]					
[13 68 8 3 5 1 4 2 19 877]]					

Test Set Evaluation:				
	precision	recall	f1-score	support
0	0.87	0.90	0.88	1000
1	0.90	0.94	0.92	1000
2	0.78	0.84	0.81	1000
3	0.77	0.61	0.68	1000
4	0.73	0.90	0.80	1000
5	0.78	0.79	0.78	1000
6	0.91	0.86	0.89	1000
7	0.91	0.78	0.84	1000
8	0.90	0.95	0.92	1000
9	0.94	0.88	0.91	1000
accuracy			0.84	10000
macro avg	0.85	0.84	0.84	10000
weighted avg	0.85	0.84	0.84	10000

Test Confusion Matrix:

```
[[900  6 19  4 12  3  2  3 45  6]
 [ 11 937  6  1  0  1  0  1 15 28]
 [ 34  0 845 25 49  7 27  8  3  2]
 [ 17  6 54 613 81 148 34 27 13  7]
 [ 14  2 32 20 895 12 10 10  4  1]
 [  7  3 40 83 47 787 10 18  3  2]
 [  4  4 35 29 38 16 864  4  5  1]
 [ 14  1 48 13 102 30  1 783  3  5]
 [ 24  9  4  2  1  2  0  2 948  8]
 [ 12 75  3  4  1  1  3  6 17 878]]
```



Overall, this meaningful increase from Phase 1 and demonstrates how proper implementation of pre-trained models, which have been understood and perfected over a longer period of time, can produce far greater results in lesser time, making it a valuable tool in machine learning. While some mistakes such as class confusion may still occur, much of the heavy lifting is taken care of from the start, allowing developers to focus on manual configuration to fix such matters.